

Software Defined Wireless Network (SDWN): an evolvable architecture for W-PANs

Salvatore Costanzo*, Laura Galluccio, Giacomo Morabito, Sergio Palazzo

Dipartimento di Ingegneria Elettrica, Elettronica ed Informatica, University of Catania (Italy)

* Department of Informatics and Telecommunications, University of Athens (Greece)

(Invited Paper)

Abstract—Rarely the interest in a new networking paradigm has increased at such a velocity as it is happening for *software defined networking* (SDN). Motivations of this hype of interest are several but the most important can be re-conducted to the following three: (i) SDN simplifies network configuration and management; (ii) SDN makes the network *evolvable*; (iii) SDN frees operators from vendors' lock-in.

Eagerly, in some cases, or reluctantly, in others, both operators and manufacturers of communication and networking equipments have started new lines of research, development, and marketing focused on SDN. A quick look at the list of members of the organizations promoting the development of SDN-related standards suffices to understand that players of the wireless domain are trying to ride this wave as well. However, it is difficult for the observers to understand where such a wave is headed to in very common networking scenarios involving infrastructureless wireless communications. This paper is an attempt to fill this gap. In fact, a major objective of this paper is to analyze (i) what are the advantages of SDN in infrastructureless wireless networking environments such as wireless personal area networks (W-PANs), and (ii) what are the requirements that must be satisfied to apply SDN in W-PANs. Furthermore, a complete SDN solution for W-PANs, named *Software Defined Wireless Network* (SDWN), is introduced and a prototype implementation is described.

I. INTRODUCTION

Software Defined Networking (SDN) clearly separates the data plane from the control plane in a network [6], [10]. The former is run by network *switches* to forward packets according to the logics set by the latter through a standard API. Accordingly, the control plane is responsible for network configuration and management which are set by a *software* program. In *OpenFlow* [8], which is currently the most popular instance of SDN, the control plane runs in a (logically) centralized manner by a dedicated server called *Controller*.

Thanks to this approach, SDN delivers a network

- which is easy to configure and manage,
- which can *evolve* because, in principle, new services and management methodologies can be introduced in the network as simply as installing a new software on a PC is,
- in which a given network node can be replaced with another produced by any vendor, so freeing the operator from the vendor lock-in.

As a result, rarely the interest in a new networking paradigm has increased at such a pace as it is happening for SDN. Most network operators are running pilot experimentations of OpenFlow networks, manufacturers are producing OpenFlow

compliant network equipment, and the research community (both academic and industrial) is engaged in a vast amount of SDN-related R&D activities. A quick look at the list of members of the *Open Networking Foundation*, an organization promoting the development of SDN-related standards, suffices to understand that this hype has spread to the wireless domain, as well.

However, to the best of our knowledge little effort has been devoted so far to the understanding of the advantages and problems which will be encountered extending the SDN approach to infrastructure-less wireless networking scenarios such as wireless personal area networks (W-PAN) [9].

Starting from our previous work [3] and the work in [5] which independently conducted a similar analysis, in this paper we first provide a comprehensive overview of the above advantages and problems.

Then, we present a complete solution that can be defined to support the SDN in W-PANs. This approach is denoted as *Software Defined Wireless Network* (SDWN). An environment, namely, the *SDWN Environment*, can be defined, which includes a prototype implementation of the SDWN protocol stack as well as a set of OMNET++ [11] simulation libraries that allow simulated nodes to interact with real elements of the SDWN. The resulting SDWN Environment can be used for fast prototyping and assessment of new management policies.

Accordingly, the rest of this paper is organized as follows. In Section II we describe the advantages of SDN in infrastructure-less wireless networks, while in Section III we discuss the requirements to be considered when applying the SDN paradigm to W-PANs. Such requirements are at the basis of the SDWN protocol stack which can be designed and prototyped as presented in Sections IV and V, respectively. Finally, in Section VI conclusions are drawn.

II. THE ADVANTAGES OF APPLYING THE SDN PARADIGM IN W-PANs

SDN does not necessarily provide advantages in terms of performance. On the contrary, SDN may even lead to some performance losses given that it envisions a higher level of abstraction and, therefore, higher overhead. Instead, as already mentioned, major advantages of SDN are related to the fact that it introduces *management simplification*, *evolvability*, and *freedom from vendor lock-in* [6], [10].

These advantages are attractive in any networking context and, thus, also in infrastructure-less wireless networks, such

as *wireless personal area networks* (W-PANs). In W-PANs the separation between the network control and management functionality and the forwarding operations offers a further advantage as discussed below.

IEEE 802.15.4 is the standard utilized for the first two layers of the protocol stack. However, the support of IEEE 802.15.4 is not sufficient to guarantee that two W-PAN nodes can interact with each other. In fact, for the higher layers of the protocol stack responsible for network control and management functionality, several alternative solutions exist (the most popular are ZigBee and 6LOWPAN) which are not compatible with each others. This is the cause of strict limitations for nodes to migrate from a network to another.

SDN solves this problem because it envisions that the network and higher layers of the protocol stack are defined through software and can be modified easily and “on the fly”.

In fact, in SDN the network management operations are centralized – at least in the earliest implementations – and physically separated from the forwarding operations. More specifically, network nodes (*SDN switches*) are responsible for packets classification according to certain *rules*, along with the corresponding *actions*. Besides, there are *Controllers* that set the rules and the corresponding actions in the SDN switches. A SDN switch that has no active rule for classifying an incoming packet, requests the help of the Controller. This will respond by sending the requesting SDN switch the rule/action pair. The rule/action pairs are set by the Controller(s) by exploiting policies (which define the *network behavior*) that can be modified easily and rapidly by *installing* a new Controller software.

As can be easily derived from the above discussion, further advantage of SDN is that it “enforces” a centralized network control. This raises crucial problems in terms of scalability and robustness but makes network optimization easier and therefore, may give advantages as will be discussed in detail in the following sections.

III. REQUIREMENTS FOR SDN SOLUTIONS FOR W-PANs

SDN solutions for traditional wired networks are expected to execute switching at line rate. Therefore, velocity has been considered as the major design requirement. Support of such a requirement has been paid in terms of low flexibility in the definition of the rules specifying the flows. For example, in OpenFlow the rules consider the typical TCP (or UDP)/IP header fields, only.

In W-PANs communications are characterized by low transmission rates, usually, and thus velocity is not a major requirement. Instead, it is extremely important to reduce energy consumption, which can be obtained in several ways [2]. Among them, the most effective relies on duty cycles, in-network data aggregation, and cross-layer optimization [4].

In the following we provide a list of some relevant requirements that must be considered in the definition of an SDN solution for W-PANs [3], [5]. The solution proposed in the following of this paper, i.e., SDWN, has been designed to specifically address such requirements.

Duty cycles must be supported – Energy consumption can be reduced by turning the radio off when possible, that

is, by using *duty cycles*. This would result in modifications of the network topology which must be taken into account when performing network control. Duty cycle management requires appropriate primitives. Accordingly, our SDWN solution envisions appropriate *actions*, as we will explain in Section V-B.

In-network data aggregation must be supported – In many W-PAN scenarios data circulating in the network is highly spatially and temporally correlated. For example, in wireless sensor network scenarios the data measured by nodes close to each other is not expected to differ significantly. The same observation applies to the data measured by the same sensor node in different time instances. Energy efficiency can be increased by reducing the redundancy from the network data traffic, that is, by using *data aggregation* techniques [7]. In SDWN an appropriate module in the protocol architecture and a new action are defined to support in-network data aggregation.

Flexible definition of the rules must be supported – SDN solutions for wired networks define rules which consider the traditional TCP (or UDP)/IP header fields only. Instead, analysis of the literature in the field of wireless sensor (and actor) networks suggests that higher flexibility is required [2], [1].

In SDWN higher flexibility is achieved in two complementary ways. On the one hand, SDWN consider rules which take any byte in the packet into account for matching purposes. This allows to treat packets based on the specific value contained in the payload they carry. On the other hand, SDWN considers rules in which matching is not necessarily conditioned by the equality between the bytes in the current packet and some reference values. In fact, in SDWN matching can be conditioned by other relational operators: for example, a route can be configured for packets in which the value contained in the payload is “higher than” a given threshold while another route is configured for the remaining packets.

Backward and peer compatibility must be guaranteed – Nodes should be able to run legacy protocol stacks so as to be used in non-SDN networks. This is required to enable new nodes to operate in existing W-PANs (*backward compatibility*). *Peer* compatibility, i.e., compatibility with wired SDN solutions, is also important as it would enable the integration of the resulting new family of software-defined W-PANs in a heterogeneous SDN network. For example, in SDWN backward compatibility is guaranteed by maintaining in the nodes a Zigbee implementation. Instead, peer compatibility could be supported by deploying an appropriate proxy. Such a proxy would strongly depend on the specific SDN solution envisioned for the wired infrastructure. Although OpenFlow is by large the most popular SDN solution, our choice has been to keep SDWN independent of the SDN solution deployed in the wired infrastructure and therefore, in its current release the SDWN Environment does not implement any proxy functionality.

There are several more obvious requirements that must be satisfied in SDWN, which will not be specifically analyzed for space constraints. Examples include the necessity to support mobility of nodes and the resulting topology changes, need to face the unreliability characterizing wireless links, and to

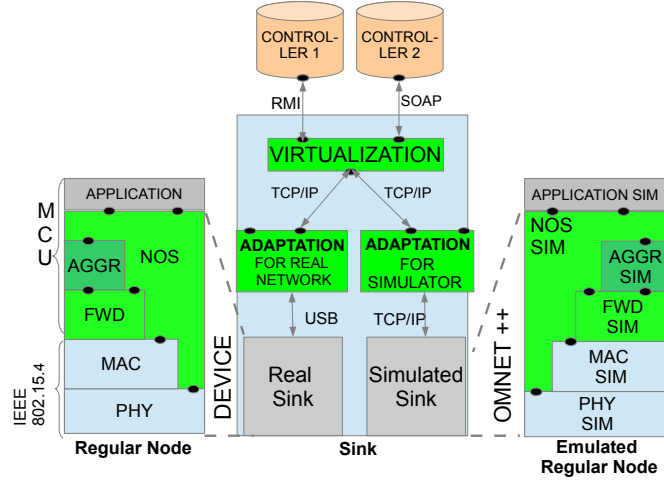


Fig. 1. SDWN protocol architecture.

withstand the failure of any node including the sink.

IV. SDWN ARCHITECTURE AND DEPLOYMENT OPTIONS

In this section we describe the protocol architecture and the typical deployment of a SDWN network. In SDWN we can distinguish several *Regular Nodes* and one (or several) *Sink(s)* which can be identified also with the WPAN coordinator(s). Sinks are the gateways between the Regular Nodes running the Data plane and the elements implementing the Control plane. The protocol stack of the Data plane run by Regular Nodes is outlined in Sections IV-A; the protocol stack of the Sink and the other elements implementing the Control Plane are described in Section IV-B; finally, Section IV-C, finally, discusses different deployment options for SDWN.

A. Data plane

The Data plane is mostly run by Regular Nodes. As shown in the left side of Figure 1, Regular Nodes include an IEEE 802.15.4 transceiver and a micro-control unit (MCU).

The IEEE 802.15.4 transceiver is configured to allow peer-to-peer communications between nodes. Above the IEEE 802.15.4 protocol stack, a **Forwarding** layer runs in the MCU which handles the arriving packets as specified in a *flow table*¹. The latter is continuously updated by the Forwarding layer according to the configuration commands sent by the Controller.

The entries of the flow tables are defined by a rule, an action, and some statistical information. A *rule* represents the characteristics of all packets belonging to a *flow*. A node identifies the flow of incoming packets according to the rules stored in the flow table and applies the *action* reported in the relevant entry. Finally, the entry contains *statistic information*, that is, the number of received packets of a given flow. More details about the implementation of the flow table will be presented in Section V-B.

Incoming packets are handed by the IEEE 802.15.4 transceiver to the Forwarding layer which classifies them according to their type in *Control* and *Data* packets. Control packets are sent to the Network Operating System (NOS) layer which will operate as described later in this section. Data packets, instead, are further processed by the Forwarding layer. More specifically, a control is executed to detect whether an incoming Data packet matches one of the rules in the flow table. If this is the case, then the Data packet is treated according to the corresponding action. Otherwise, the Data packet is given to the NOS layer.

The **Aggregation** layer runs on top of the Forwarding layer and is responsible for aggregating the data flowing through the network. Our current implementation of SDWN defines the *aggregation equivalent flow* (AEF) through a set of rules stored in one of the flow table entries. The Data packets belonging to a given AEF are sent to the Aggregation layer which concatenates them, if possible.

Concerning the **NOS** layer, it is the heart of the SDWN protocol stack at regular nodes, and can access all layers of the protocol stack by means of appropriate APIs. Thus, it can gather local information from the nodes and control their behavior at all layers, according to the indications provided by the Controller.

To this aim, the NOS layer must be able to interact with the Controller at any time. SDWN applies a proper scheme to this purpose, as explained in Section V-A.

The NOS layer provides an API to the application layer as well, which extends the IEEE 802 APIs. This guarantees legacy with existing applications.

Note that the same protocol suite is run by the Regular Nodes modeled in the OMNET++ simulator, as plot in the right hand side of Figure 1.

B. Control plane

In the Control plane, the network management logics are dictated by one or several Controller(s). In fact, thanks to the Virtualization layer which abstracts the network resources, it

¹We derive our terminology from OpenFlow [8].

is possible to run different logical networks, with different management policies, over the same set of devices. Each coexisting network is governed by a different Controller and is characterized by a rule which is used to identify packets. Each packet satisfying such rule will be treated according to the policies set by the corresponding Controller. The Controllers may run either in the same node hosting the Virtualization layer or in different nodes. As a consequence, the interactions between the Controllers and the Virtualization layer can occur in several ways, as shown in the central part of Figure 1. In fact, in the case the Controllers run in the same node hosting the Virtualization layer, interactions will occur through the Java methods offered by the Virtualization layer. In the opposite case (i.e., Controller and Virtualization layer run in different nodes), the interactions can occur through the Java *remote method invocation* (RMI) or the *Simple Object Access Protocol* (SOAP). In this way, programmers can implement Controllers either in Java or some Web programming languages.

The Virtualization layer exploits information collected by the NOS layer running in the Regular Nodes to build a consistent representation of the current network status (topology, levels of energy of all Regular Nodes, SNR characterizing different links, etc). In SDWN the network topology is represented through a Java Map. The map contains the following information regarding a Regular Node, A :

- Last time the Sink has received a packet generated by A .
- Level of battery reported in the last Control packet received from A .
- A list of the neighbors of A . For each neighbor, say B , the Java Map reports the address (currently this is a 2 byte field), the quality of the link² between A and B , and the last time the Controller has received a packet from A that reports B among the neighbors of A .

For the Map, a timeout is defined which is the time after which an entry (or part of it) should be removed if there are no evidences enforcing a different action.

The Virtualization layer interacts with two Adaptation modules, one is connected to a real Sink, the other is connected to a simulated Sink. The Adaptation modules format the messages so that the Regular Nodes can handle them. Concerning the Sinks, note that they implement all functions run by Regular Nodes (both “real” and “simulated”).

C. Deployment

As already discussed in Section IV-A, Regular Nodes consist of an IEEE 802.15.4 transceiver and a MCU. For example, in our experimental testbed Regular Nodes are implemented on TI eZ430-RF2500 devices which contain a TI CC2500 transceiver and TI MSP430 micro-control unit.

As regards the Control plane, the simplest deployment option is the one depicted in Figure 2(a), in which the node hosting the Sink is the same where the Virtualization layer and the Controllers run. The above option requires the presence of

²The link quality is measured in terms of RSSI and is represented through 1 byte.

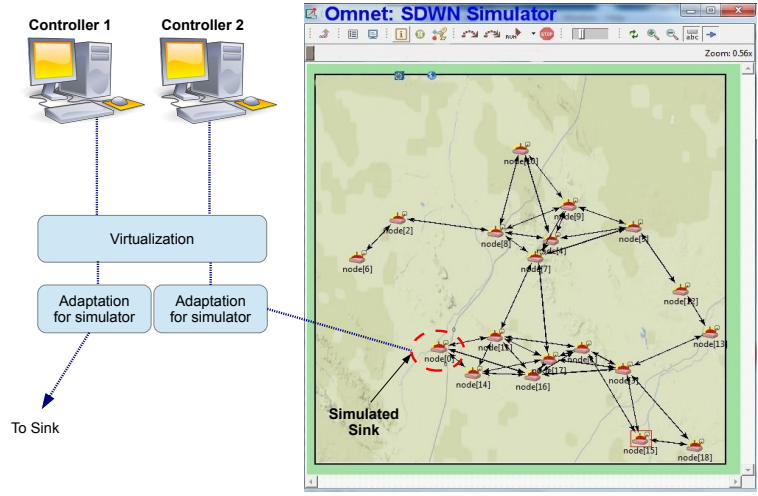


Fig. 3. Integration with the OMNET++ simulator.

a node with significant computational resources in the area where the Regular Nodes are deployed.

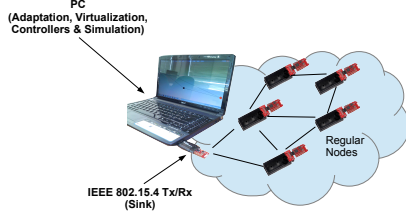
In several scenarios, however, it is not possible to deploy such powerful nodes in the network area. In these cases, the Sink is usually attached to an embedded system that accesses the Internet through some communication interface. For example, in our experimental testbed represented in Figure 2(b), the Sink is a TI CC2500 device attached via USB to a Beagleboard running a Linux operating system. The Adaptation layer is implemented in the Beagleboard which sends control packets to the Virtualization layer running on a remote server. In our testbed the Beagleboard is equipped with an UMTS interface (the smartphone in Figure 2(b)) and communication between the Adaptation and Virtualization layers occurs through TCP/IP connections.

The Controllers may be hosted by other PCs (or virtual machines) and interact with the Virtualization layer in several ways. In our testbed we support both SOAP and RMI interaction models.

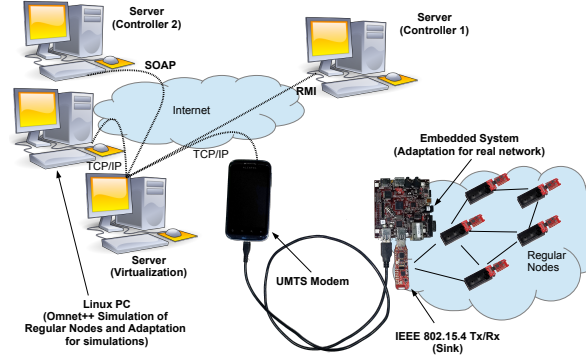
Finally, simulations modeling the behavior of the Regular Nodes and the Simulated Sink can be executed on another PC. In Figure 3 we show a screenshot from an OMNET++ simulation showing the current topology of the network composed by Regular Nodes. Node 0 is the Sink and interacts through the Adaptation module with a real instance of the Virtualization layer. Accordingly, Controllers 1 and 2 can be real controllers determining the policies which are utilized by the simulated Regular Nodes.

V. DESIGN AND IMPLEMENTATION

In this section we present some features of the SDWN implementation. In fact, in Section V-A we describe the operations executed by the NOS layer to support the interactions between a Regular Node and the Controller. In Section V-B we provide detailed specifications of the representation of rules and actions. Finally, in Section V-C the format of relevant SDWN packets is presented.



(a) Centralized control plane.



(b) Distributed control plane.

Fig. 2. SDWN deployment options.

A. Collection of topology information

The NOS layer is responsible for the collection of the topology information needed by Regular Nodes to interact with the Controller.

To this aim, the sink periodically sends a *beacon* packet throughout the network in multi-hop. At each hop the beacon packet contains information about the current distance from the sink (in terms of number of hops) and a measure of the local battery voltage. Accordingly, upon receiving a beacon packet a Regular Node increases the value of the current distance from the sink by one, overwrites the value of the current level of the battery, and then, forwards the packet. Furthermore, while receiving a beacon packet the Regular Node also measures the current RSSI. This way, each Regular Node gets the information that can be used to identify the most convenient next hop to reach the Sink. In particular, firstly, it favors the node that declares the lowest distance from the Sink, secondly, the node with the longest battery life and, thirdly, the node toward which the highest RSSI value was measured.

To perform the above selection, each Regular Node stores the list of neighbors (with the corresponding RSSI) in a specific table, named *neighbors table*. Periodically the table is sent to the Sink using an appropriate *report packet*, which will be forwarded to the best next hop identified as explained above.

The Sink will receive the list of neighbors from all Regular Nodes and will use this information to infer the network topology.

B. Specification of rules and actions

Regular Nodes have strict limitations in terms of memory, therefore a memory-efficient methodology must be identified to define rules. In order to limit the space required to store the rule in the flow table, SDWN rules operate only upon a limited portion of the incoming packet. In particular, a rule operates on (up to) three windows of at most two bytes of the incoming packet. Examples of rules stored in the flow table are reported in Table I.

Observe that each flow table entry contains three blocks (*window blocks*) related to three windows of bytes that define a rule, one block (*action block*) that specifies the action that

TABLE I
EXEMPLARY FLOW TABLE

Window I				Window II				...	Action		Stats
Size	Op	Addr	Value	Size	Op	Addr	Value		Type	Value	
2	=	2	170.24	2	≠	4	170.11	..	Forward	170.23	17
2	=	2	170.16	1	=	1	3	..	Drop	1	3
2	≠	2	170.24	1	=	7	25	..	Modify	7/26	3
2	=	2	170.17	0	=	0	0	..	Forward	170.21	11

must be performed on the packets of the flow, and a *counter* used for statistical information.

Each window block includes the following information:

- size: it gives the number of bytes of the window. It can assume a value in the range (0, 2). If it is set to zero, then the window is ignored.
- operator: it defines the relational operator that must be checked to consider the matching valid. In the current SDWN release, we implement the operators “=”, “≠”, “<”, and “>”.
- address: it gives the position of the first byte of the window.
- value: it gives the value which must be matched against the byte(s) of the window.

Upon receiving a packet, the matching algorithm in the Forwarding layer checks whether each window of the packet matches the value stored in the field “value” and, if so, it performs the action that is given in the “Action/Type” field.

The type of action can be “forward”, “modify”, “drop”, “aggregate”, and “turn off radio”. The action will be performed according to the value contained in the “Action/Value” field. This field has length equal to two bytes and assumes a different significance depending on the type of action:

- If the action is “forward”, the two bytes of the “Action/Value” field are used to identify the next hop.
- If the action is “modify”, the first byte of the “Action/Value” field is used to identify the specific byte of the packet which must be modified and the second byte to indicate the value to be assigned to such byte.
- If the action is “drop”, the first byte is used to give the

bit byte	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	Packet Length								Network ID							
2	Source Address															
4	Destination Address															
6	Type of Packet								Time To Live							
8	Address Next Hop															

Fig. 4. Packet format.

probability with which the packet should be dropped (by default this probability is set to one). The second byte indicates the last byte of the address of the node to which the packet should be forwarded in case it is not dropped. Note that if there are several nodes with equal last byte in their address, the node will choose randomly (this is implementation specific) the one to which the packet will be actually forwarded.

- If the action is “aggregate” the two bytes are used to identify the next hop to which the aggregated packet must be forwarded.
- If the action is “turn off radio” the two bytes of the “Action Value” field are used to specify the duration of the interval during which the radio interface must be off.

For example, the first entry in the in Table I specifies that packets that have in bytes 2 and 3 the values 170 and 24 and that do not have in bytes 3 and 4 the values 170 and 11 must be forwarded to node 170.23. Finally, the last value in the line specifies that 17 packets have been classified according to this rule so far.

The flow table entry will be populated with the values contained in the “Rule/action Response” packets arriving to the Sink.

C. Packet format

All packets circulating in the network use a fixed header that is organized as shown in Figure 4. Referring to Table I discussed at the end of previous section, observing that:

- bytes 2 and 3 represent the source address;
- bytes 4 and 5 represent the destination address;

the first flow table entry can be interpreted as follows: all packets generated by node 170.24 and that are not directed to node 170.11 must be forwarded to node 170.23.

Additional fields may be included based on the type of packet:

- **Type 0 - Data packet:** This packet is generated/delivered by/to the application layer and contains the payload, as well as the header as depicted in Figure 4.
- **Type 1 - Beacon packet:** This is the packet which is broadcast periodically by the Sink. Such a packet contains the header and an additional byte (referred to as “Hops to sink”) providing the number of hops required to reach the Sink from the node which has *transmitted* the packet.
- **Type 2 - Report packet:** This packet is generated by a Regular Node upon receiving a beacon packet and is

transmitted to the Sink in multi-hop. Besides the header, Report packets contain the “Hops to sink” byte, another byte reporting the current battery charge level, and the list of the current neighboring nodes.

- **Type 3 - Rule/action Request:** This packet is generated by a node upon receiving a packet which does not match any of the rules stored in the flow table. It is generated as a copy of the incoming packet in which, however, the Type of Packet field (i.e., byte 6 in the header) is set to 3 and the original Type of Packet value is inserted at byte 10.
- **Type 4 - Rule/action Response:** Besides the usual header this packet contains a pair “Rule/Action” as described in Section V-B. More specifically, this type of packet contains information about the window size (2 bit), the relational operator (3 bit), the position in the packet of the first byte of the window (8 bit), and the “value” field (16 bit).
- **Type 5 - Open path packet:** This type of packet is used to build a path using a single packet. Accordingly, it contains the usual header followed by a Rule and the sequence of the addresses of the nodes in the path³.

VI. CONCLUSIONS

In this paper we have comprehensively discussed the advantages and challenges of applying SDN in wireless networks. Also, we have presented the requirements and the guidelines that are relevant to the design of an SDWN architecture for W-PANs by illustrating those we have used in our implemented testbed solution, named SDWN. Furthermore, we have outlined several alternative options for the SDWN deployment. Our prototype implementation has been developed to offer SDWN programmers a wide range of options on the programming language to use. Finally, SDWN offers a fast prototyping environment exploiting the OMNET++ simulator.

REFERENCES

- [1] K. Akkaya and M. Younis. A survey on routing protocols for wireless sensor networks. *Ad Hoc Networks*. Vol. 3, No. 3. Mar. 2005.
- [2] I. F. Akyildiz, W. Su, Y. Sankarasubramanian, E. Cayirci. A survey on sensor networks. *IEEE Comm. Magazine*. Vol. 40, No. 8. Aug. 2002.
- [3] S. Costanzo, L. Galluccio, G. Morabito, and S. Palazzo. Software Defined Wireless Networks: unbridling SDNs. *EWSDN 2012*. Oct. 2012.
- [4] F. Cuomo, and A. Abbagnale, and E. Cipollone. Cross-layer network formation for energy-efficient IEEE 802.15.4/ZigBee Wireless Sensor Networks. *Ad Hoc Networks* Vol. 11, No. 2, 2013.
- [5] T. Luo, H.-P. Tan, and T. Q. S. Quek. Sensor OpenFlow: Enabling Software-Defined Wireless Sensor Networks. *IEEE Communications Letters*. Vol. 16, No. 11. Nov. 2012.
- [6] N. McKeown, T. Anderson, H. Balakrishnan, G. Parulker, L. Peterson, J. Rexford, S. Shenker, and J. Turner. OpenFlow: Enabling Innovation in Campus Networks. *White paper*. Mar. 2008.
- [7] S. Palazzo, F. Cuomo, and L. Galluccio. Data aggregation in wireless sensor networks: a multifaceted perspective. in *Sensor Networks: Where Theory Meets Practice* (G. Ferrari, Ed.), Springer, 2009.
- [8] B. Pfaff *et al.*. OpenFlow Switch Specification – Version 1.1.0 Implemented (Wire Protocol 0x02). Feb. 2011.
- [9] J. Qadir, N. Ahmed, and N. Ahad Building Programmable Wireless Networks: An Architectural Survey. <http://arxiv.org/abs/1310.0251>. Oct. 2013.
- [10] S. Shenker *et al.*. The future of networking and the past of protocols. *OpenNetSummit 2011*. Oct. 2011.
- [11] <http://www.omnetpp.org/>

³Our current implementation does not support “Open path” packets.